

CANDY DIALOGUE ENGINE

NPC Barks

1.0.0

Adding NPC barks requires making use of one of Candy DE's advanced features: parallel dialogues.

Luckily, this guide explains how to set this up very simply.

The first thing to understand is that each instance of the **Candy_Engine.gd** script acts as a dialogue engine of its own. This means two or more instances of the script can run at the same time, each using their own UI (to avoid conflicts), and this is key to NPC barks.

UI

For NPC barks, you'll only need a minimal UI. And luckily, one is already made: Default.tscn in Candy_DE/Scenes/Barks_2D or in Candy_DE/Scenes/Barks_3D (depending if you are displaying barks in a 2D or 3D environment).

Simply add this Bark UI to your character nodes (e.g. sprites or meshes, or whatever represents them in game space), anywhere in their node structure: the important thing is that each character has its own Bark UI.

The provided Bark UIs consist of a root node named "BarkEngine", with the **Candy_Engine.gd** script attached (the same as the Candy_UI scene).

The Bark UI features a "BarkBubble" node, which will display dialogue in a manner similar to speech bubbles. It also features an AudioStreamPlayer ("VoicePlayer") for playing voiced dialogues, in case your NPC Barks are voiced. "BarkBubble" has either the **Bark_Bubble_2D.gd** or **Bark_Bubble_3D.gd** script attached.

After assigning the Bark UI to a character node, you'll need to position the UI (or at least the Bark node) so that it is located where you want it relative to the character node it belongs to.

Scripts

Candy_Engine.gd

As mentioned, the Bark UI has a copy of **Candy_Engine.gd** attached. If you look in the Inspector, some settings are configured for NPC barks specifically. For example, notice that the default **dialogue_mode** is "Bark".

Note that **input_enabled** is set to false, so that player input won't interfere with Barks (such as skipping them).

Since in most cases Barks should automatically continue (or end) without player input, **auto_advance** is set to 3.0.

Most paths in **ui_elements_paths** and **media_players_locations** have been removed: the Bark UI scene normally doesn't need most of the UI elements that the Candy_UI scene uses.

bubbles_npc_path and **bubbles_player_path** are empty, because the Bark UI isn't supposed to target and display speech bubbles on other character nodes: it uses its own speech bubble.

Feel free to modify any other settings as you wish.

If you want barks to be voice only (no text), just switch **dialogue_mode** to "Voice".

Bark_Bubble2D/3D.gd

The **Bark_Bubble_2D.gd** or **Bark_Bubble_3D.gd** scripts are identical to **Speech_Bubble_2D.gd** and **Speech_Bubble_3D.gd**. The reason they are provided as different scripts is in case you want to customize the code of Bark Bubbles and regular Speech Bubbles differently from one-another.

For information on these scripts, see the **Speech Bubbles** guide.

Bark Bubbles vs Speech Bubbles

As mentioned, Bark Bubbles and Speech Bubbles, by default, use the same code. They also share the same node structure.

The difference between Bark Bubbles and Speech Bubbles is how they are used by **Candy_Engine.gd**:

Speech Bubbles are used by the "Bubbles" dialogue mode (or "VN Bubbles" in VN mode), while Bark Bubbles are used by the "Bark" dialogue mode.

The "Bubbles" mode can display Speech Bubbles on any character node: it locates the node of the speaking character and, if the node exists, it displays text in the Speech Bubble attached to the character. This works fine because the "Bubbles" dialogue mode only makes one character speak at a time.

NPC Barks, however, are meant to occur at any moment, in the background, even when normal dialogue is occurring. Multiple NPCs may emit barks at the same time, independently of one-another, hence why they each need their own instance of the dialogue engine (i.e. **Candy_Engine.gd**).

In "Bark" mode, the dialogue engine has only one job: to display dialogue for the character it's assigned to, nobody else.

As such, the Bark Bubble acts as a hybrid of a Speech Bubble and Dialogue Box: it can change its size to adapt to the text it's displaying, but it's part of the engine's UI. The Bark Bubbles is only

attached to the character node indirectly, because the engine node its a child of is itself attached to the character node).

Hence, in **Candy_Engine.gd**, the path of Bark Bubbles is assigned to a key in the **ui_elements_paths** dictionary (like the Dialogue Box and most other UI elements), instead of being assigned to **npc_bubbles_paths**.

If this is confusing, here's a simple way to think about it: imagine that a Speech Bubble, instead of being assigned to a character node, is made a child of the Candy_UI scene, as an alternative to the dialogue box. Then imagine that Candy_UI is duplicated into a new instance, and this instance is assigned as a child of a character node instead of being an autoload. That's what Bark UIs and Bark Bubbles are.

Triggering Barks

Your game will need to be programmed to trigger barks as appropriate.

That means your game must do two things:

1. Determine which NPC should emit a Bark.
2. Determine what the NPC should say.

How you detect which NPCs emit barks is up to you. You may want to base this on their proximity to the player, and set a timer or some randomness logic to ensure they don't speak too often.

To determine what NPCs should say, you have several options:

- a) Create different dialogues entirely for each NPC (NPCs with the same bark lines can share the same dialogue).
- b) Create a single 'Bark' dialogue, where each NPC has a different Conversation or Block for their barks.

Barks, of course, are Spoken Lines in dialogues. Once you've chosen either of these two methods, you only need to do one more thing: write a bit of code that determines which bark text to speak (i.e. which Conversation, Block and Line to start dialogue at).

Once your code has determined all of that, it just needs to call `start_dialogue()` on the BarkEngine node of the correct NPC, passing the correct Conversation/Block/Line as arguments.

The Candy_Enging.gd script attached to that NPC will start a dialogue, run the lines, then end -- as normal.

When writing bark Spoken Lines, remember that roles can be assigned as the speaker instead of specific actors. Whether you should use roles depends entirely on your implementation, but it's worth keeping in mind as it might make things easier.

Shared UI

You can make NPCs display their barks in a shared UI, such as a chatbox (where each new bark line is added after the previous one).

This requires careful implementation and is not recommended, unless you're experienced with Candy DE and Godot, and you have a specific need for this. We won't provide detailed explanations, but here are some tips:

An important thing to do is to set **writing_speed** to 0 so that barks display instantly. Otherwise, if the typewriter effect is enabled, NPCs producing barks at the same time will fight over the chatbox and cause conflicts.

Alternately, if you are determined to display barks in a shared UI with typewriter effect, you should program your game to ensure only one NPC ever outputs a bark at a time, or you may need to create your own custom chatbox UI and possibly a custom dialogue mode for it.

In any case, if you want NPCs to use a shared UI, be sure to modify the **"barks_path"** key in **ui_elements_paths**, in **Candy_Engine.gd**: simply assign the path to the shared UI element you want to use.

Barks UI Customization

Bark bubbles are identical to speech bubbles. To customize them, see the **Speech Bubbles Customization** guide.